

## 質問コーナー

## 第1回 Q&amp;A まとめ

300件超の質問から厳選した12テーマ。  
すべてに共通するメッセージは——



この図解は**要点をまとめたもの**です。具体例やニュアンスなど、より詳しい内容は**動画本編**をご覧ください。

## AI時代の武器 = 「めんどくさい」を見つける力

ツールは変わる。原則は変わらない。

  
**めんどくさいを発見**  
超神経質モードで探す



  
**AIで武器化**  
ルールを守って作る  
Git + スキル + SSoT



  
**自分だけの武器**  
仕事で使えるツール

12問を貫くキーワード

 Git

 スキル

 SSoT

 パッチワーク禁止

 繰り返して身につく

ここから先で、12の質問と回答をひとつずつ図解していきます。

## 全12問インデックス

**01 Cursor vs Claude Code**

使い分けの基準

**02 衝撃を受けたAIサービス**

Claude Coworkの可能性

**03 AIアイデアが浮かばない**

「めんどくさい」を見つける力

**04 スキルの育て方**

チューニングと整理整頓

**05 SSoTとリポジトリ管理**

情報整理のファーストステップ

**06 ツール開発の手順**

AI-Driven Rulesに尽きる

**07 デザイン崩れ防護策**

Git + パッチワーク禁止

**08 AIとセキュリティ**

情報の入力判断基準

**09 トークン費用の考え方**

AI時代の「税金」

**10 ChatGPTとCursorの併用**

シームレスな学びの流れ

**11 チームへのAI導入**

動くツールが最強の説得力

**12 ドリルの進め方**

立ち止まらず薄く重ねる

**01 Cursor vs Claude Code の使い分け**

どのような用途で使い分けていますか？

**<> Cursor — 7割**

- ✓ コンテンツを作りながら**ツールを磨く**作業に最適
- ✓ **仮想環境**やSlack連携が便利
- ✓ **初心者はずまずCursor**がおすすめ

💡 プログラミング以外の用途にも強い

**>\_ Claude Code — 3割**

- ✓ **アプリ開発**が決まっていればこちら
- ✓ **プログラミングに最適化**された設計
- ✓ Web/スマホアプリを**最初から作る**時に使う

</> プログラムを作ることに特化

### 🔗 マコのひとこと

「どちらかが圧倒的に良いということではなくて一長一短あります。初心者のうちはCursorでいいんじゃないのかというのが私の理解です」

## 02 衝撃を受けたAIサービス

Cursor以外で衝撃的だったサービスは？

答え

### Claude Cowork

パソコン上でやりたいこと、ほぼすべてできる



#### ブラウザ操作

Claude in Chromeで賢くブラウジング



#### ファイル操作

パソコン内のファイルを自在に扱える



#### シームレス

難しいこともCoworkに任せられる



#### ただし

AIの時代の流れは早いので、どのツールが便利かは**常に変化**します。現状ではCursorの価値もGitの価値もまだまだある。Coworkにも限界はあります。

### 🔗 マコのひとこと

「パソコン上でやりたいことほぼ全部できちゃう。でもAIの時代の流れは早いので、常にキャッチアップしていくことが大事です」

## 03 AIを仕事に活かすアイデアが浮かばない

AIを仕事に活かしたいけどアイデアが浮かばない

### アイデア発見の3ステップ

1



#### 超神経質モードになる

自分の気質を忘れて  
「めんどくさい」に  
敏感になる

2



#### 小さく作って見せる

個人プロジェクトで  
「こんなことができますよ」  
を見せる

3



#### 決裁者の心を動かす

仕組みそのものを  
変えられるように  
働きかける

#### 🔗 マコのひとこと

「アイデアを見つけるのがうまい人は、めんどくさいを見つけるのがうまい人です。すべての職場においてAIを活用して自動化する余地なんて無限にあるはずなんです」

## 04 スキルの育て方・管理方法

使いながらアップデートとは具体的にどうするのか？

## スキルを育てるサイクル

### 1 違和感に気づく

「色が派手」「文章が長い」など、繰り返し見て発見する

### 2 具体例で伝える

キャプチャやリンクを添えて「ここが長い」と具体的に指示

### 3 本質を捉えて改善

「パッチワークではなく本質を捉えて」と一文を添える

### 4 自分で読んで整理

重複・古い情報を自分の目で確認し整理整頓する

🔄 1→2→3→4 を繰り返して精度を上げる

## 💡 レビューのコツ

「最近のスキルについて熟知した**Anthropicのエンジニア**としてサブエージェントを立ち上げてレビューをしてください」と指示する。スキルを企画した側の視点でレビューしてもらう。

## 📖 マコのひとこと

「チューニングって言って細かい修正をしていくんですけど、自分で読むことが大事なんです。AI が書いた文章を自分で読んで整理整頓する」

↑ インデックスに戻る

## 05 SSoTとリポジトリ管理

全社の情報をどう分割・管理すればいい？

## リポジトリの構成イメージ

会社全体

### 📁 事業A リポジトリ

- アプリコード
- CS情報
- 全社共通/

### 📁 事業B リポジトリ

- Webサービス
- マーケ資料
- 全社共通/

### 📁 経営・機密 リポジトリ

- 給与情報
- 評価面談
- 権限を厳格管理

🔄 全社共通フォルダはGitHub Actionsで自動同期

### 🗨️ マコのひとこと

「どんなツールを使おうともSSoTの原則を厳守するということは変わりません。いいツール・いいやり方は常に変わるけど、SSoTは守るべきということは変わりません」

### 💡 実践ルール

最初は1事業部1リポジトリで始める。機密情報は別リポジトリに分離。大きくなりすぎたら適宜切り出す。全社共通情報はソース元フォルダから自動配信する。

## 06 ツール開発の手順

バチッとはまる感覚が得られない。手順はある？

× **期待: 「たった1つのコツ」で全部うまくいく**

✓ **現実:**

プロサッカー選手に「絶対うまくなる1つのコツは？」と聞いたら「**練習することだ**」としか答えようがない。全12回のプログラムを通じて、いろんな切り口で伝え続けるのがこの講義の設計。

× **不安: AIの書いたコードが全部理解できない**

✓ **安心してください:**

現役のエンジニアもすべては理解できていない。大枠を理解し、ディテールのディテールまで全部読んでいる人は今ほとんどいない。学習を積み重ねれば「バチッとはまる感じ」はいつか必ず来る。

## 📖 マコのひとこと

「開発の手順があるのか。これはAI-Driven Rulesを厳守して作っていくに尽きます。配布した図解ツールの中身をAIに噛み砕いて解説してもらってほしいです」

## 07 デザイン崩れ防護策

AIに微修正を頼むとスタイルが崩れる。どう防ぐ？

答え

### Git + パッチワーク禁止

こまめにコミット。修正は全体を見て行う。

**⊗ NG: パッチワーク修正**

- × 「ここもっと右」「この色変えて」の**繰り返し**
- × AIが**部分だけに集中**し全体を見失う
- × 修正するたびに**他の場所が壊れる**

**☑ OK: プロの仕事**

- ✓ 「**プロのUI/UXデザイナー**として仕事して」
- ✓ 修正後のUIを**アスキーアート**で**事前確認**
- ✓ こまめに**Gitでコミット**して戻れるように

**🔗 マコのひとこと**

「一番良くないのはパッチワークを続けること。パッチワークを繰り返していくとAIも何に集中していいかわかんなくてどんどん性能が落ちていきます」

**08 AIとセキュリティ**

どこまで情報を入れていいのか？

**情報の種類別 判断ガイド****一般的な業務情報・公開情報**

基本的にAIに入力OK

**顧客データ・営業資料**

個人情報をサンプルデータに置換 / 社名を伏せて入力

**個人情報・機密情報そのまま**

名前・メール・住所・電話番号をそのまま入力しない

## 📖 マコのひとこと

「危険か危険じゃないかという二元論ではなく、個別具体の情報でこれは良いのか悪いのかを判断していくのが大事。守るものが多ければより保守的に、少なければ車は危ないから乗らないという意味決定をする必要はない」

## 💡 大事な前提

ほとんどのAIツールは**学習に使うかどうかの設定**ができる。また大手AI企業は個人が特定されない形に加工していると各社は言っている。ただし**詳しくは必ず自分で確認**すること。会社所属している場合は会社のルールに従う。

[↑ インデックスに戻る](#)

## 09 トークン費用の考え方

Cursorの利用枠の減りが想定以上。どう抑える？

**5,000円+**

AIを十分に活用する  
最低ラインの月額費用

**数万円**

ガチで使うなら  
月数万円も不思議ではな  
い

**= 税金**

AIの利便性を手にすると  
もう使わない選択肢がな  
い



### 出力量を減らす

AIのアウトプットの量を減らすことでトークン消費を抑える



### 軽いモデルを使う

リーズナブルなモデルで済む作業はそちらに切り替える

## 📖 マコのひとこと

「ジャブジャブトークンを使えるのが常に正義ではありません。トークン消費を節約するというのはAIにとって効率的な指示の出し方、仕事のやらせ方を考えるに等しい行為だからです」

## 10 ChatGPTとCursorの併用

ChatGPTとCursor、両方使っても大丈夫？

結論

**ChatGPT Plusで構いません**

ただしCursorにはCursorの強みがある

なぜCursorが便利か



調べ物



即座に反映



Gitで保存

調べる → 反映する → 保存する がシームレスに繋がる

### 🔗 マコのひとこと

「仕事に関連する調べ物をしたらその調べ物のデータを保存したくなります。保存する最良の方法はリポジトリでGitでコミットすることです。なのでCursorが便利なんです」

## 11 チームへのAIツール導入

「保守はできるのか」という抵抗にどう対応する？

## 説得の戦略

3

### 感情で納得させる

「これいいね!」と感情が動けばコストも許容される

2

### 動くものを見せる

言葉の説明より、実際に動くツールのデモが最強

1

### まず小さく作る

社内の一部の人が使う小さなツールから始める

下から順に積み上げていく

## 🔗 マコのひとこと

「うわこれすごい良いねと思えるようなツールを作るに勝る説得力はないってことです」

## 12 ドリルの進め方

ドリル後半が難しく立ち止まってしまう。進めて大丈夫？

× **不安: 全部理解しないと先に進めない**

✓ **結論: 大丈夫です**

ドリルは**何度も薄く重ねる**設計。1回ですべて理解できることはない。前に進み続けることで自然と復習もでき、知識が繋がっていく。

## 💡 一番大事なこと

ドリルはあくまでドリル。実際に自分の作りたいものを作るという「自分で作ってみる」に勝る学びは存在しません。

### 📖 マコのひとこと

「何度も薄く重ねていってまだわかっていないんだけどなって思っても、前に進み続けることで自然と復習もできて応用に進んでいけるように設計しています」

[↑ インデックスに戻る](#)

---

AI-Driven School の図解ツールで作成